PWA Offline Behaviour Specification — Family Music Scrobbler PWA

PWA Offline Behaviour Specification - Family Music Scrobbler PWA

A complete reference describing how the Family Music Scrobbler Progressive Web App behaves when offline, how it caches data, how it synchronises queued actions, and how it maintains a seamless user experience across unreliable network conditions.

📌 1. Purpose of This Document

This specification defines:

- Offline caching strategy
- IndexedDB storage model
- Background Sync behaviour
- Service Worker responsibilities
- UI behaviour when offline
- Recovery and reconciliation logic

The goal is to ensure the PWA remains fully usable even when the device has no network connection.

🧱 2. Offline Architecture Overview

The PWA uses three core technologies:

1. Service Worker

Handles caching, routing, and offline fallbacks.

2. IndexedDB

Stores cached API responses and offline sync queue.

3. Background Sync API

Automatically retries queued requests when connectivity returns.
Together, these create a robust offline-first experience.

🌐 3. Caching Strategy

3.1 Cache-First (App Shell)

The following assets are cached on first load:

- HTML shell
- CSS
- JavaScript bundles
- Fonts
- Icons
- Manifest

This ensures the app loads instantly even when offline.

3.2 Network-First (API Data)

For dynamic data such as:

- Recent tracks
- Leaderboard entries
- Profile data

The PWA attempts a network request first, then falls back to cached data.

3.3 Stale-While-Revalidate

Used for:

- Track metadata
- Album art

This returns cached data immediately and updates it in the background.

🗄️ 4. IndexedDB Storage Model

IndexedDB stores two categories of data:

4.1 Cached API Responses

Tables:

```
recent_tracks
leaderboard
profile_cache
```

4.2 Offline Sync Queue

Stores user actions performed offline:

```
sync_queue
```

Each entry contains:


```
{
  "id": "uuid",
  "endpoint": "/users/me",
  "method": "PATCH",
  "payload": { ... },
  "timestamp": "2026-09-09T07:46:00Z"
}
```

🔄 5. Background Sync Behaviour

5.1 Queueing Logic

If a network request fails due to connectivity:

- The request is stored in sync_queue
- The UI immediately reflects the change (optimistic update)

5.2 Sync Trigger

When the device reconnects:

- Service Worker triggers a sync event
- Each queued request is replayed in order
- Successful requests are removed from the queue

5.3 Retry Strategy

- Exponential backoff
- Maximum retry count
- Error logging

📡 6. Service Worker Responsibilities

6.1 Install Event

- Cache app shell
- Precache static assets

6.2 Fetch Event

- Route API calls

- Apply network-first or cache-first strategies

6.3 Sync Event

- Replay queued requests

6.4 Push Event

- Display leaderboard notifications

6.5 Activate Event

- Clean old caches

📱 7. UI Behaviour When Offline

7.1 Offline Indicator

Header displays:

```
Status: Offline
```

7.2 Disabled Actions

Actions requiring immediate network access are disabled:

- Spotify relink
- Manual ingestion trigger

7.3 Optimistic Updates

Profile edits and settings changes:

- Apply instantly
- Sync later

7.4 Offline Pages

The following pages work fully offline:

- Home
- Recent Tracks (cached)
- Leaderboard (cached)
- Profile (cached)
- Settings

🔁 8. Recovery & Reconciliation

8.1 Sync Conflicts

If server data differs from offline edits:

- Server version wins
- User notified via toast

8.2 Cache Refresh

After reconnection:

- All cached API responses are refreshed
- UI updates automatically

8.3 Failed Sync Items

If an item fails permanently:

- Marked as failed
- User can retry manually

🧪 9. Testing Offline Behaviour

9.1 Simulated Offline Mode

Use browser devtools:

- Disable network
- Inspect service worker logs

9.2 IndexedDB Tests

Verify:

- Queue insertion
- Queue replay
- Cache reads

9.3 PWA Install Tests

Ensure:

- App loads offline
- Manifest is valid

🏁 10. Summary

This offline behaviour specification ensures the Family Music Scrobbler PWA:

- Works reliably without network access
- Caches essential data
- Syncs user actions automatically
- Provides a seamless user experience
- Recovers gracefully from connectivity issues

It forms the foundation for a robust, offline-first music analytics application.